

PROBLEMA 1. Verificar si un arreglo está ordenado.

```
< > Problema1.java
... Problemas Problema 1 Problema1.java
1 public class Problema1 {
2
3     public static boolean ordenado(int[] arr, int n) {
4         if (n <= 1) {
5             return true;
6         }
7
8         if (arr[n - 2] > arr[n - 1]) {
9             return false;
10        }
11
12        return ordenado(arr, n - 1);
13    }
14
15    public static void main(String[] args) {
16        int[] ejemplo1 = {1, 2, 3, 5, 8};
17        int[] ejemplo2 = {1, 4, 2, 8};
18        System.out.println("Resultado 1: " + ordenado(ejemplo1, ejemplo1.length));
19        System.out.println("Resultado 2: " + ordenado(ejemplo2, ejemplo2.length));
20    }
21 }
22 |

~/Documentos/ExamenParcial main*
> cd ~/Documentos/ExamenParcial/Problemas/Problema\ 1/

~/Documentos/ExamenParcial/Problemas/Problema 1 main*
> java Problema1
Resultado 1: true
Resultado 2: false

~/Documentos/ExamenParcial/Problemas/Problema 1 main*
> |
```

Análisis Teórico

Explicación del Caso Base:

El caso base es nuestra condición de parada para que el programa no se quede en un bucle infinito como vamos restándole 1 al tamaño n en cada pasada, cuando n llega a ser 1 o menor significa que ya no hay más parejas que comparar. Un arreglo con un solo número o vacío siempre está ordenado, por eso devolvemos true

Obtener T(n) :

En cada llamada hacemos una comparación condicional, después, la función se vuelve a llamar a sí misma una sola vez reduciendo el tamaño del problema en un elemento (n - 1) cumple con: $T(n) = T(n - 1) + c$

Analizar la complejidad temporal.

Segun la tabla de patrones de recursion el modelo $T(n - 1)$ tiene una complejidad de crecimiento lineal por lo tanto es $O(n)$

PROBLEMA 2. Segundo elemento más grande usando divide y vencerás

```
< > Problema2.java
Problemas Problema 2 Problema2.java
1 public class Problema2 {
2
3     public static int[] buscarDosMayores(int[] arr, int izq, int der) {
4
5         if (izq + 1 == der) {
6             if (arr[izq] > arr[der]) {
7                 return new int[]{arr[izq], arr[der]};
8             } else {
9                 return new int[]{arr[der], arr[izq]};
10            }
11        }
12
13        if (izq == der) {
14            return new int[]{arr[izq], arr[izq]};
15        }
16
17        int medio = (izq + der) / 2;
18        int[] resultadoIzq = buscarDosMayores(arr, izq, medio);
19        int[] resultadoDer = buscarDosMayores(arr, medio + 1, der);
20        int maxGlobal;
21        int segGlobal;
22
23        if (resultadoIzq[0] > resultadoDer[0]) {
24            maxGlobal = resultadoIzq[0];
25
26            if (resultadoIzq[1] > resultadoDer[0]) {
27                segGlobal = resultadoIzq[1];
28            } else {
29                segGlobal = resultadoDer[0];
30            }
31        } else {
32            maxGlobal = resultadoDer[0];
33
34            if (resultadoDer[1] > resultadoIzq[0]) {
35                segGlobal = resultadoDer[1];
36            } else {
37                segGlobal = resultadoIzq[0];
38            }
39        }
40
41        return new int[]{maxGlobal, segGlobal};
42    }
43
44    public static int obtenerSegundo(int[] arr) {
45        if (arr.length < 2) {
46            return -1;
47        }
48        int[] resultado = buscarDosMayores(arr, 0, arr.length - 1);
49        return resultado[1];
50    }
51
52    public static void main(String[] args) {
53        int[] arreglo = {8, 3, 10, 5, 7};
54        System.out.println("Segundo mas grande: " + obtenerSegundo(arreglo));
55    }
56
57    ~/Documentos/ExamenParcial/Problemas/Problema 2 main
58    > java Problema2
59    Segundo mas grande: 8
60
61    ~/Documentos/ExamenParcial/Problemas/Problema 2 main
62    > []
```

```
< > Problema2.java
Problemas Problema 2 Problema2.java
30 segGlobal = resultadoDer[0];
31
32 }
33 } else {
34     maxGlobal = resultadoDer[0];
35
36     if (resultadoIzq[0] > resultadoDer[1]) {
37         segGlobal = resultadoIzq[0];
38     } else {
39         segGlobal = resultadoDer[1];
40     }
41 }
42
43 }
44
45 return new int[]{maxGlobal, segGlobal};
46 }
47
48 public static int obtenerSegundo(int[] arr) {
49     if (arr.length < 2) {
50         return -1;
51     }
52     int[] resultado = buscarDosMayores(arr, 0, arr.length - 1);
53     return resultado[1];
54 }
55
56 public static void main(String[] args) {
57     int[] arreglo = {8, 3, 10, 5, 7};
58     System.out.println("Segundo mas grande: " + obtenerSegundo(arreglo));
59 }
60
61 ~/Documentos/ExamenParcial/Problemas/Problema 2 main
62 > java Problema2
63 Segundo mas grande: 8
64
65 ~/Documentos/ExamenParcial/Problemas/Problema 2 main
66 > []
```

Análisis Teórico

Explicar cómo se combinan los resultados.

Usando la lógica de Dividir, Vencer y Combinar que vimos en la Búsqueda Binaria la parte de combinar junta las soluciones de las dos mitades, comparamos los ganadores de cada lado para ver cuál es el máximo de todo el arreglo. Sabiendo quién ganó, el segundo lugar general sale de comparar al máximo perdedor con el segundo lugar de la mitad ganadora usando unos if.

Obtener T(n) :

Dividir: toma constante O(1) porque solo saca el medio

Vencer hace 2 llamadas recursivas por nivel procesando los datos n/2

Combinar: usa condiciones if else directos sin los bucles por lo que cuesta tiempo constante

La ecuación sería $T(n) = 2T(n/2) + c$

Analizar la complejidad temporal. Big O

Segun la tabla de patrones de recursion el modelo $T(n - 1)$ tiene una complejidad de crecimiento lineal por lo tanto es $O(n)$

Usando la formula general $T(n) = aT(n/b) + f(n)$ donde $a = 2$, $b = 2$ y $f(n) = c$

Hojas : $n^{\log_2 2} = n$

Cuando comparamos estas dos partes del trabajo de las hojas n aplasta el trabajo constante de combinación c . por lo que es lineal $O(n)$

PROBLEMA 3. Contar inversiones en un arreglo

```
< > Problema3.java
Problemas Problema 3 Problema3.java
1 public class Problema3 {
2
3     public static int mergeSortYContar(int[] arr, int izquierdo, int derecho) {
4         int conteo = 0;
5
6         if (izquierdo < derecho) {
7             int medio = (izquierdo + derecho) / 2;
8             conteo += mergeSortYContar(arr, izquierdo, medio);
9             conteo += mergeSortYContar(arr, medio + 1, derecho);
10            conteo += fusionarYContar(arr, izquierdo, medio, derecho);
11        }
12    }
13
14    return conteo;
15 }
16
17 private static int fusionarYContar(int[] arr, int izquierdo, int medio, int derecho) {
18     int[] temp = new int[derecho - izquierdo + 1];
19     int i = izquierdo;
20     int j = medio + 1;
21     int k = 0;
22     int inversionesCruzadas = 0;
23
24     while (i <= medio && j <= derecho) {
25         if (arr[i] <= arr[j]) {
26             temp[k++] = arr[i++];
27         } else {
28             temp[k++] = arr[j++];
29             inversionesCruzadas += (medio - i + 1);
30         }
31     }
32
33     while (i <= medio) {
34         temp[k++] = arr[i++];
35     }
36
37     while (j <= derecho) {
38         temp[k++] = arr[j++];
39     }
40
41     for (int x = 0; x < temp.length; x++) {
42         arr[i + x] = temp[x];
43     }
44
45     return inversionesCruzadas;
46 }
47
48 public static void main(String[] args) {
49     int[] arr = {3, 1, 2};
50     int total = mergeSortYContar(arr, 0, arr.length - 1);
51     System.out.println("Resultado: " + total);
52 }
53 }
54
~/Documentos/ExamenParcial/Problemas/Problema 3 main
> java Problema3
Resultado esperado: 2
~/Documentos/ExamenParcial/Problemas/Problema 3 main
> []
```

```
< > Problema3.java
Problemas Problema 3 Problema3.java
26 temp[k++] = arr[i++];
27 } else {
28     temp[k++] = arr[j++];
29     inversionesCruzadas += (medio - i + 1);
30 }
31 }
32
33 while (i <= medio) {
34     temp[k++] = arr[i++];
35 }
36
37 while (j <= derecho) {
38     temp[k++] = arr[j++];
39 }
40
41 for (int x = 0; x < temp.length; x++) {
42     arr[i + x] = temp[x];
43 }
44
45 return inversionesCruzadas;
46 }
47
48 public static void main(String[] args) {
49     int[] arr = {3, 1, 2};
50     int total = mergeSortYContar(arr, 0, arr.length - 1);
51     System.out.println("Resultado: " + total);
52 }
53 }
54
~/Documentos/ExamenParcial/Problemas/Problema 3 main
> java Problema3
Resultado esperado: 2
~/Documentos/ExamenParcial/Problemas/Problema 3 main
> []
```

Análisis Teórico

Explicar cómo se detectan las inversiones.

La inversión se encuentra en el método merge cuando los ordenamos, si el número de la izquierda es mayor que la de la derecha existe una inversión. Como las dos mitades ya están ordenadas por la recursión, si $arr[i]$ es mayor significa que todos después de él hasta medio también serán mayores y para no hacer otro bucle los contamos juntos restando las posiciones $medio - i + 1$.

Obtener $T(n)$:

Hace lo mismo que mergesort se dividen en dos llamadas recursivas que procesan la mitad de los elementos y el método merge recorre todo el arreglo de forma lineal para acomodar los números: $T(n) = 2T(n/2) + n$

Analizar la complejidad temporal.

Sacamos los datos $a = 2$, $b = 2$, y el trabajo de combinación es $f(n) = n$

Hojas: $n^{\log_2 2} = n^1 = n$

Si comparamos veremos que es exactamente igual al trabajo de combinación por lo que estamos en empate perfecto. Su regla dice que cuando hay un empate la complejidad final se saca multiplicando ese término por la altura del árbol que sería $\log n$: $O(n \log n)$

Por qué el algoritmo es más eficiente que revisar todos los pares posibles:

Si lo hacemos con dos bucles for anidados, tendríamos que comparar cada número con los demás. Si tenemos 1 millón de elementos haría muchas operaciones por lo que no sería correcto con divide y vencerás se va achicando a la mitad cada nivel que generamos y las operaciones baja.